Ensemble Learning and Random Forests

1. What You'll Learn in This Section

In this lesson, you'll learn to:

- Explain how ensemble learning combines multiple models to make a more reliable decision.
- Describe how a random forest builds many decision trees using two sources of randomness.
- Compare a single decision tree against a random forest on accuracy, stability, and feature importance.
- Apply the bootstrap sampling and feature randomness rules to build your own intuition about why random forest works.

2. Detailed Explanation

Ensemble learning: many models, one decision

Ensemble learning is not a brand-new algorithm. It is a strategy for improving predictions by combining multiple models that already exist, basing the final decision on their collective output instead of trusting a single model.

Picture a binary classification problem: labeling an email as spam or not spam. Three separate models are trained on the same data:

- Model 1 — Logistic Regression
- Model 2 — Support Vector Machine (SVM)
- Model 3 — Decision Tree

For one email, Model 1 predicts spam, Model 2 predicts not spam, and Model 3 predicts spam. The final call is made by majority voting, so the email is classified as spam.

The intuition: if one model makes a mistake by chance, the other models can still be correct, and the majority vote overrides the mistake. If all three models agree, there's no benefit and no harm — the same class gets assigned either way. This kind of ensemble (independent models voting) generally does not degrade the output a single model was already producing, because each extra model brings additional information. In most cases, an ensemble of independent classifiers gives a modest but reliable improvement — typically not a drastic jump, but around a two to three percent improvement over any one model alone.

Random forest: an ensemble built entirely from decision trees

Random forest works differently from the "combine three different algorithms" ensemble above. Instead of mixing algorithm types, random forest is built entirely on top of the decision tree algorithm — it creates many decision trees and takes a majority vote (or an average, for regression) among them. Its internal design still makes it an ensemble by construction.

Machine learning algorithms can broadly be divided into:

- Individual learning algorithms — for example, logistic regression, SVM, decision tree.
- Ensemble-based learning algorithms — derived from a fundamental learning algorithm; random forest belongs here.

Scikit-learn's module structure reflects this exact distinction: `DecisionTreeClassifier` lives in the `tree` module, while `RandomForestClassifier` lives in the `ensemble` module.

Why a single decision tree struggles

A decision tree's biggest strength is that it is highly explainable: because splits are chosen using the Gini index, you can point to exactly why a feature became the root node (it had the highest Gini index), and the same logic explains every split down to the leaf.

That explainability comes at a cost — a single decision tree is rigid. Whichever attribute produces the highest Gini index at the root always becomes the root, no matter what. Train one tree on a large dataset (say, 10,000 instances) and test it on new instances (say, 1,000), and that tree cannot adapt. With enough data and several available features, this rigidity leads to overfitting. The tree performs very well on training data but poorly on unseen test data, because its decision-making is locked into one fixed, Gini-driven structure.

Watch out for: decision trees are still genuinely useful on small datasets — even around 10 instances is enough to build one. Algorithms like logistic regression, SVM, and neural networks typically outperform a single decision tree, but only once they have much larger datasets to learn from.

Two sources of randomness that build a forest

The fix for a rigid, overfitting tree is to build multiple decision trees instead of relying on one tree trained on all instances and all features. Each tree is trained slightly differently, so together they capture different aspects of the same data. Once more than one tree is involved, the setup becomes a random forest. Two independent sources of randomness make the individual trees as different from each other as possible:

Feature randomness (per split). At each split, in each tree, a decision tree in a random forest does not consider every available feature — it randomly selects a subset. The convention: randomly select a number of features equal to the square root of the total number of features available at that level. For example, if 16 features are available, 4 are

randomly chosen for that split (one of those 4 wins based on Gini index); if 4 features are available, 2 are chosen. The selection is purely random, not based on any notion of "best" features — because different trees receive different random feature subsets at each level, no two trees end up with identical structure.

Bootstrap sampling (row randomness). Each tree in the forest trains on a resampled dataset of the same size as the original, but the rows are chosen through bootstrap sampling: picked randomly, with replacement. "With replacement" means a selected data point is placed back into the pool afterward, so it stays eligible to be picked again. The resulting sample is the same size as the original but can contain repeated rows and can be missing some original rows entirely.

Suppose the original dataset `X` has six data points: `a, b, c, d, e, f`. Bootstrapped training samples for three trees might look like this:

Tree	Sample drawn (with replacement)	Notes
DT1	`c, c, f, d, f, d`	`c`, `f`, `d` repeated; `a`, `b`, `e` not selected
DT2	`e, e, a, f, a, d`	shares `f` and `d` with DT1 by chance, otherwise different
DT3	`b, b, c, c, a, e`	another independent draw

Each tree trains on a dataset the same size as the original but composed differently — some rows repeated, others absent — purely by chance. This process is called bootstrapping. Two bootstrap samples don't have to be identical, nor are they

guaranteed to differ — but with a reasonably large dataset, the odds of getting the exact same sample twice are very low.

Why bootstrap sampling is trusted, not biased

A natural question: if sampling is done with replacement and repeats are possible, doesn't that introduce bias? An analogy helps here. Imagine an instructor has 100 problem statements to hand out to 100 student groups. If the instructor reads each problem statement before assigning it, personal judgment can creep in — which group is "good at math," which group the instructor likes or dislikes. That's where real bias appears, even with good intentions. But if the instructor assigns problem statements completely randomly, without reading them first, the process is trusted precisely because no judgment is involved. Even a biased instructor can't act on that bias if the assignment mechanism itself is blind and random.

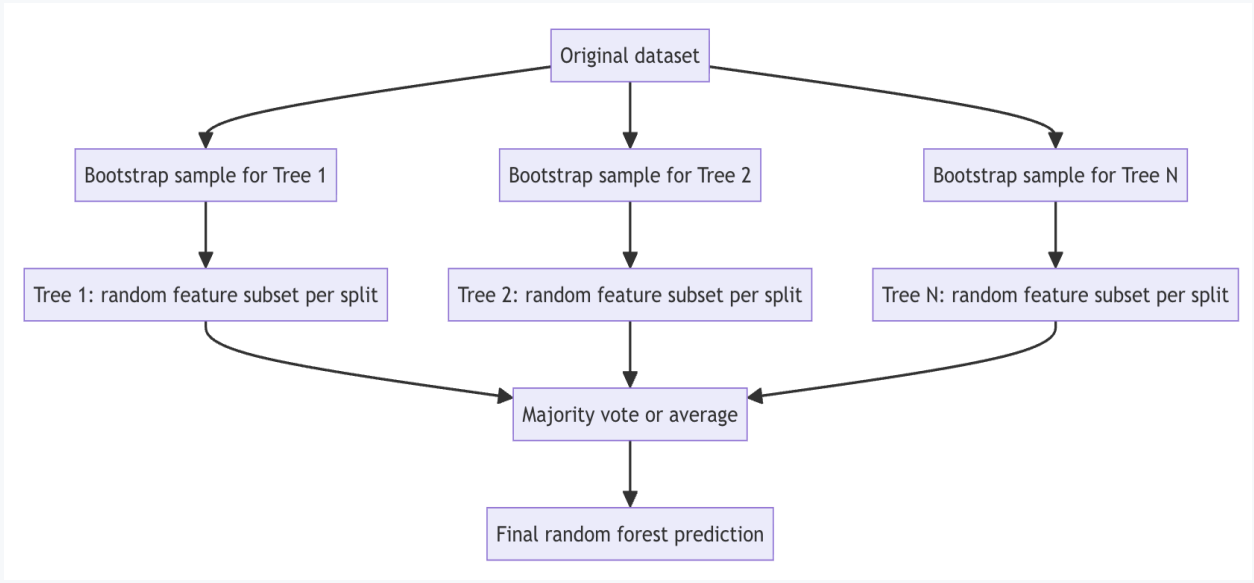
The same logic applies to bootstrap sampling: theoretically it introduces no systematic bias, because rows are chosen without any criteria. By pure chance, two samples could occasionally turn out similar, or one tree could get an unusually "hard" sample — but that's chance, not bias, and it's compensated for by having many trees rather than one.

Combining the trees: voting, averaging, and the tree-count hyperparameter

Once multiple decision trees are built — each from a different bootstrap sample of rows, each choosing a different random feature subset at every split — the forest combines their outputs:

- Classification: the final prediction is decided by majority voting across all trees.
- Regression: the final prediction is the aggregation (average) of all trees' predicted values. For example, if five regression trees each predict a numeric score, the final output is the sum of all five predictions divided by five.

The number of trees in the forest (for example, 5, 10, 15, 20, or 100) is a hyperparameter. Different values get tried, and whichever produces the best performance on the given dataset is selected for delivery. This implementation detail — the exact number of trees used — is typically not disclosed to the client in full, but it should still be recorded internally (for example, in a README file) so future developers or maintainers understand what was implemented.



A human analogy: guessing what Gandhi would say

The logic behind ensembles mirrors how humans make collective decisions. When a decision affects a group of people, opinion isn't taken from just one person, because one person's view may be shaped by limited background or fixed assumptions.

Concretely: suppose five people are each given a book about Mahatma Gandhi to read, then asked what Gandhi's likely reaction would have been to a contemporary situation, such as a paper-leak scandal. Each person, having read different material, brings a different partial understanding. Collecting their opinions and taking the majority produces an answer considered more reliable than any one person's, because it draws on more diverse knowledge.

A refined version of the analogy maps more precisely onto random forest: instead of five different books, imagine one collection of five books is handed out, but each of the five people randomly receives only one book from that collection. The overall pool of source material is the same, but each person's specific exposure is randomized. This mirrors random forest exactly — every tree is drawn from the same original dataset, but each is randomly restricted to different rows (via bootstrapping) and different feature subsets (via feature randomness). The trees end up "slightly different" rather than "completely different," since they're all derived from the same underlying data, just arranged differently through randomization.

Worked example: predicting employee attrition

An HR team wants to predict which employees are likely to leave the company (attrition), using historical data with four real features:

- Tenure (years) — higher tenure generally means lower chance of leaving.
- Satisfaction score — a value between 0 and 1; low satisfaction is a strong signal of leaving.
- Monthly overtime hours — higher overtime is treated as a signal toward leaving.
- Salary percentile — higher percentile generally reduces the chance of leaving, though this varies: some employees are content with a modest salary if their workload feels proportionate.

These weightings describe majority-case tendencies used to build training labels, not universal rules — the relative importance of each feature can vary from person to person.

A single decision tree trained on this historical data performs reasonably well on its training data. But when tested on a slightly different sample of similar employees — for example, a new sample of 1,000 employees after training on 10,000 from one office, or

employees from a different office — performance degrades noticeably. The reason: relying purely on Gini-index splits makes the tree overly sensitive (overfit) to its specific training data.

The fix is the same one seen earlier: draw multiple bootstrap-sampled decision trees from the same employee data and combine them by majority voting — a random forest. Using the earlier six-letter toy example (`a, b, c, d, e, f` standing in for six employee records), a bootstrap sample might be `a, f, a, d, c, d`. Here `b` and `e` are not selected for that particular tree, which is fine because other trees are highly likely to include them.

Building and testing the models in code

The comparison starts with standard imports. Notice that `RandomForestClassifier` comes from `sklearn.ensemble` while `DecisionTreeClassifier` comes from `sklearn.tree` — confirming scikit-learn's own classification of random forest as an ensemble algorithm:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

A synthetic employee dataset of 150 instances is generated with 10 columns: `tenure_years` (0–15), `satisfaction_score` (0–1), `monthly_overtime_hours` (0–30), `salary_percentile` (0–1), five noise columns (`noise_0` through `noise_4`, random values 0–100), and a `left` label (1 = left, 0 = stayed). The noise columns are deliberately added to test whether the algorithms correctly assign them little to no importance. This

mirrors a real-world case where a client provides data without knowing in advance which features are actually predictive.

A linear combination `z` of the four real features is computed, with each feature given a signed weight reflecting its relationship to attrition. Tenure and satisfaction reduce the chance of leaving (negative weight), overtime hours increase it (positive weight), and salary percentile also reduces it (negative weight, roughly -2). Satisfaction was given a comparatively large-magnitude coefficient (about 4, negative in direction) to reflect it being treated as the dominant factor — chosen intuitively rather than derived from a fixed formula.

The resulting `z` is passed through the sigmoid function to get a probability of leaving between 0 and 1. Rather than using a fixed cutoff (which would force every employee with the same probability to get the same label), each employee's probability is compared against an independently generated random number. If the probability is higher, the label is 1; otherwise it is 0. This introduces realistic randomness so identical feature values don't force identical outcomes. Printing `y.mean()` on the resulting labels gives an attrition rate of 21%.

A separate, standalone cell demonstrates bootstrapping mechanics directly on the six-letter toy array:

```
tiny = np.array(['a', 'b', 'c', 'd', 'e', 'f'])
np.random.seed(2)
bootstrap_sample = np.random.choice(tiny, size=len(tiny), replace=True)
print(list(bootstrap_sample))
```

With the seed fixed at 2, this produced `['a', 'f', 'a', 'd', 'c', 'd']` — `b` and `e` were never selected, while `a` and `d` each appear twice. The `replace=True` argument is what allows a chosen element to stay available for future draws — the "with

replacement" behavior. When using `RandomForestClassifier` directly, this bootstrap sampling happens internally and automatically; it never needs to be coded explicitly.

Comparing accuracy: decision tree vs. random forest

A helper function measures accuracy across multiple random train/test splits of the same data:

```
def accuracy_score_split(model_fn, X, y, n_splits=6):
    accuracy_array = []
    for rs in range(n_splits):
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.3, random_state=rs
        )
        model = model_fn()
        model.fit(X_train, y_train)
        preds = model.predict(X_test)
        accuracy_array.append(accuracy_score(y_test, preds))
    return accuracy_array
```

This function is called once for a decision tree and once for a random forest, using `lambda` to pass a fresh, uninstantiated model constructor rather than a single pre-built model instance:

```
tree_accuracy = accuracy_score_split(lambda:
DecisionTreeClassifier(random_state=42), X, y)
rf_accuracy = accuracy_score_split(
    lambda: RandomForestClassifier(n_estimators=100, random_state=42), X, y
)
```

Without the `lambda`, the same model object could get reused and incrementally modified across splits instead of a completely fresh model being trained each time. Wrapping the constructor in `lambda` guarantees every call to `model_fn()` inside the loop

creates a brand-new tree (or a brand-new 100-tree forest) trained from scratch on that split's training data. The random forest is configured with `n_estimators=100`, and `test_size=0.3` is used for every split.

Across all six splits, the decision tree's accuracy is highly volatile, ranging from as low as 0.62 up to 0.88, while the random forest stays consistently higher and far more stable:

Model	Mean accuracy	Spread (max − min)
Decision tree	≈ 0.71	≈ 0.26 (about 26 percentage points)
Random forest	≈ 0.81	≈ 0.11 (about 11 percentage points)

That's roughly a 10-point improvement in mean accuracy, and less than half the variability, when switching from a single tree to a 100-tree forest on the same data. In terms of variability, the single decision tree is described as being "twice as much in error" compared to random forest. A scatter plot puts the tree's six accuracy scores at `y = 1` and the forest's six scores at `y = 0`, on the same accuracy axis. It shows the random forest's minimum accuracy already exceeds most of the tree's individual scores — the tree's scores are far more spread out (roughly 0.6 to 0.88) than the forest's narrower, higher band. This pattern — random forest outperforming a single decision tree with far less variance — holds "in general," reported consistently across data domains, including finance, for both classification and regression tasks.

Feature importance: what each model treats as important

A fresh single split (`test_size=0.3`, `random_state=0`) trains a `DecisionTreeClassifier` and, separately, a

`RandomForestClassifier(n_estimators=100, random_state=42)` on the same training data. Both expose a `feature_importances_` attribute, compared side by side:

Feature	Single decision tree	Random forest
tenure_years	0.136	0.147
satisfaction_score	0.288 (highest for both)	high, but noticeably lower than the tree's score
monthly_overtime_hours	second-highest	second-highest, but relaxed relative to the tree
salary_percentile	effectively 0 — the tree captured no signal here	some positive, non-zero importance captured
noise columns	some non-zero importance assigned (spuriously)	some non-zero importance assigned (spuriously)

Two conclusions follow. First, random forest is more "relaxed" than a single decision tree. The single tree assigns a very high, rigid importance score to whichever feature dominates (satisfaction, here) and can effectively ignore other useful signals, like salary percentile. Random forest, by voting across 100 trees, distributes importance more evenly and captures signals the single tree missed entirely. A follow-up bar chart (green bars for random forest, red for the single tree) visually reinforces this: the tree's

importance is spiky and concentrated, while the forest's is smoother and more generalized.

Second, neither algorithm inherently filters out noise features. Both models assign some small importance to the artificially injected noise columns, simply because those columns exist and show some accidental correlation with the target by chance. Removing noise features requires a separate, explicit feature-selection step, such as `SelectKBest` with an ANOVA-based scoring function, which selects the top-scoring features and excludes the rest. Only about half of the "leaving" decision — roughly the four real features — is actually meaningful; the remaining contribution technically leaks in from the noise columns, so removing them should visibly improve performance further.

Random feature selection is not "best" feature selection

Random forest's per-split feature selection has nothing to do with picking the "best" features. `SelectKBest` explicitly scores every feature against the target using an ANOVA-based score that effectively measures correlation between feature and output, then keeps only the top-scoring ones. Random forest's feature randomness is the opposite: at each split, the subset of available features is chosen with no scoring, no "best," and no "worst" — purely randomly. This is why the algorithm is named "random" forest: the randomness is deliberate and unweighted, not merit-based.

Computational complexity

Random forest is more complex to implement than a single decision tree, since it must repeat the bootstrap-sampling-plus-feature-randomness process for every tree in the forest (for example, 100 times). The base decision tree algorithm itself is not very computationally expensive — fundamentally a set of if-else rules driven by Gini index comparisons. Because of this, random forest remains far less computationally

expensive overall than learning-based algorithms such as logistic regression, SVM, or neural networks. This makes it a low-cost, high-benefit substitute for a single decision tree in almost every applicable case.

3. Key Takeaways

- Ensemble learning combines multiple models and decides by majority vote, typically giving a modest 2–3% improvement over any single model.
- Random forest is an ensemble built entirely from decision trees, using two randomization sources: bootstrap sampling of rows (with replacement) and random feature subsets at each split (sized by the square root of available features).
- A single decision tree is explainable but rigid, which leads to overfitting; random forest trades some of that explainability for far more stable, reliable accuracy.
- Random forest aggregates via majority voting for classification and averaging for regression, with the number of trees (`n_estimators`) as a tunable hyperparameter.
- Neither a single tree nor a random forest automatically filters out noise features — that still requires an explicit step like `SelectKBest`.

Mental model: Think of random forest as a group of readers who each get a different, randomly-restricted slice of the same book collection — individually limited, but collectively far more reliable than any one reader's opinion.